

calc3 — Project Architecture

A learning calculator in C3: tokenizer, recursive descent, AST and tree traversal via visitors.

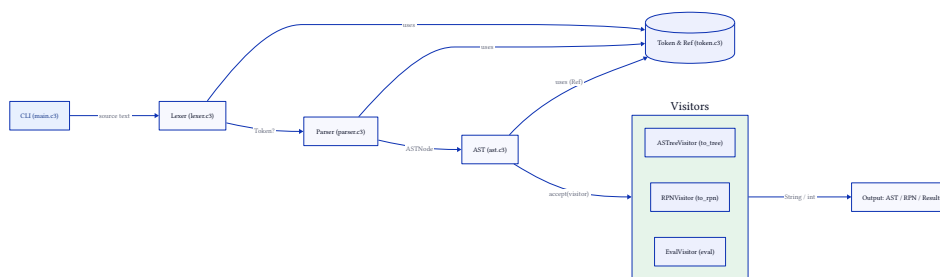
Overview

The program takes the text of an arithmetic expression and produces three representations: the AST (tree), RPN (reverse Polish notation) and the computed value. The data flow is the classic compiler pipeline:

source -> Lexer -> Parser -> AST --(accept)--> Visitors -> output

All stages share the position type `token::Ref { row, col }`, so error messages carry source coordinates.

Data flow



Modules

Module	Responsibility
token.c3	Token types (TokenKind), token value (Value), position (Ref) and a shared error-formatting helper <code>format_ref_error</code> .
lexer.c3	Turns source text into a token stream (<code>Lexer.next</code>), tracks <code>Ref</code> , and catches unknown characters and integer overflow.
ast.c3	Tree nodes (<code>ASTNumber</code> , <code>ASTUnary</code> , <code>ASTBinary</code>), the <code>ASTNode/ASTVisitor</code> interfaces, factories and a demo <code>ASTTestVisitor</code> .
ast_tree.c3	The <code>ASTreeVisitor</code> visitor + <code>to_tree</code> — renders the AST as a tree (in the style of <code>eza -T</code>).
rpn_visitor.c3	The <code>RPNVisitor</code> visitor + <code>to_rpn</code> — emits reverse Polish notation (NEG/POS for unary operators).
eval_visitor.c3	The <code>EvalVisitor</code> visitor + <code>eval</code> — evaluates the value, returning <code>Result{int, EvalError}</code> .
parser.c3	Recursive descent over the grammar (<code>parse</code>), builds the AST, and reports <code>ParseError</code> with position.
main.c3	Entry point: CLI argument parsing, the REPL, file reading and assembling the final output.

Token representation (token.c3)

- `TokenKind` is an enum: `NUMBER`, `PLUS`, `MINUS`, `MUL`, `DIV`, `MOD`, `LPAREN`, `RPAREN`, `EOF`, plus the fault values `UNKNOWN_CHAR`, `OVERFLOW`.
- `Token` is `{ kind, val: Value, ref: Ref }`, where `Value` is a union (number or character).
- `Ref` is `{ int row, int col }`, the single source of error coordinates.

- `format_ref_error(String kind, Ref ref, String msg)` is a shared template, "`<kind>(<msg> at <row>:<col>)`", used by both `ParseError` and `EvalError`.

Lexer (`lexer.c3`)

- Lexer holds input, ref and pos; `eat` updates ref (including newline and tab handling).
- `next()` -> `Token?`: skips whitespace, dispatches on the first character via `switch` to emit a token; for digits it calls `eat_int` (catching overflow). On error it returns a fault (`UNKNOWN_CHAR~ / OVERFLOW~`) rather than throwing an exception.

AST and the Visitor pattern (`ast.c3`)

- The `ASTNode` interface requires `accept(ASTVisitor)` and `to_format` (for `Printable`).
- The `ASTVisitor` interface has `visit_number / visit_unary / visit_binary`.
- Nodes: `ASTNumber { ref, value }`, `ASTUnary { ref, op, operand }`, `ASTBinary { ref, op, left, right }`. Each `accept` calls its corresponding `visit_*`.
- The factories `create_number / create_unary / create_binary` allocate a node and return it as `ASTNode`. `op_symbol` maps an operator `TokenKind` to its symbol.
- `ASTTestVisitor` is a demo visitor that prints node structure (used in a test).

Visitors

Each visitor implements `ASTVisitor` and accumulates its result in a `DString out` (or, for `EvalVisitor`, in `int? value / EvalError? error`):

- `ASTreeVisitor` → `to_tree(ASTNode, Allocator) -> String` — a tree with indentation and box-drawing.
- `RPNVisitor` → `to_rpn(ASTNode, Allocator) -> String` — postfix notation.
- `EvalVisitor` → `eval(ASTNode, Allocator) -> Result{int, EvalError}` — the expression's value.

All three return an **owning** copy of the string (`str_view().copy(allocator)`), so the caller does not hold a dangling reference to the visitor's internal buffer.

Parser (`parser.c3`)

- `parse(String src, Allocator allocator) -> Result{ASTNode, ParseError}` — recursive descent: `expression` → `term` → `factor` → `primary`, honoring precedence and associativity, with support for unary `+/-` and parentheses.
- `ParseError` holds a `Ref` and a message; its `to_format` delegates to `token::format_ref_error("ParseError", ...)`.

Error handling

Two independent error types, each with a `to_format`:

- `ParseError` — position and cause at the parsing stage (e.g. `ParseError(Unexpected character at 1:4)`).
- `EvalError` — position and cause at the evaluation stage (Division by zero, overflow including `INT_MIN / -1`).

The shared `token::format_ref_error` guarantees a consistent format and the presence of coordinates.

Entry point and CLI (`main.c3`)

The `pipeline(String src, Allocator) -> Result{PipelineOk, ParseError}` combines `parse` + `to_rpn` + `eval` and returns the node, RPN and result in a single pass. On top of it:

- `process_line(src, allocator)` — a short RPN\nresult output (used in tests); the input ? answers 42.
- `process_to_string(src, allocator)` — the full AST / RPN / Result block.
- `process(src)` — prints the `process_to_string` result inside a pool (@pool + tmem).

CLI argument parsing is factored into `parse_args` (it fills the state through pointer out-parameters and reports whether to continue), while the modes live in `process_file` (reads a file, splits it into lines, processes each) and `run_repl` (an `io::treadline` loop over `stdin`). `main` stays a thin dispatcher:

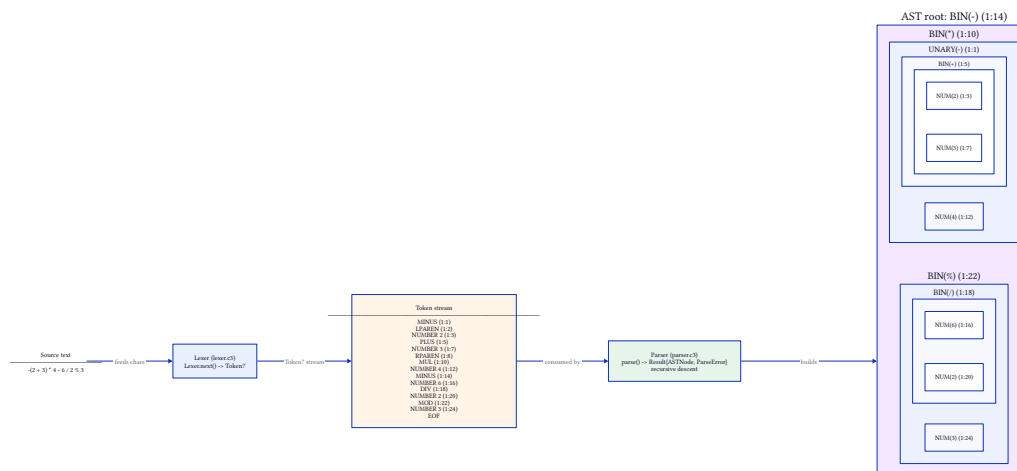
- `calc3` — interactive REPL;
- `calc3 "<expression>"` — a single expression;
- `calc3 -f <file> / --file <file>` — expressions read from a file (one per line);
- `calc3 -h / --help` — show help;
- `--` — a separator to pass an expression that starts with `-` (e.g. `-5`).

Allocator and ownership

Mid-level functions (`parse`, `to_tree`, `to_rpn`, `eval`, `pipeline`) take an `Allocator` and return owning copies. The top level (`process`, `process_line`, `process_to_string`) runs inside an `@pool` with the temporary allocator `tmem`, which simplifies freeing memory after the output is built.

Example: from text to AST

Let's analyze the expression from the tests: `-(2 + 3) * 4 - 6 / 2 % 3`.



The lexer turns the source into a token stream (each with its own Ref): MINUS (1:1), LPAREN (1:2), NUMBER 2 (1:3), PLUS (1:5), NUMBER 3 (1:7), RPAREN (1:8), MUL (1:10), NUMBER 4 (1:12), MINUS (1:14), NUMBER 6 (1:16), DIV (1:18), NUMBER 2 (1:20), MOD (1:22), NUMBER 3 (1:24), EOF.

The parser (recursive descent) builds the tree:

```

BIN(-) (1:14)
├── BIN(*) (1:10)
│   ├── UNARY(-) (1:1)
│   │   └── BIN(+) (1:5)
│   │       ├── NUM(2) (1:3)
│   │       └── NUM(3) (1:7)
│   └── NUM(4) (1:12)
└── BIN(/) (1:18)
    ├── NUM(6) (1:16)
    └── BIN(%) (1:22)
        └── NUM(3) (1:24)
  
```

```

└─ BIN(%) (1:22)
  └─ BIN(/) (1:18)
    └─ NUM(6) (1:16)
      └─ NUM(2) (1:20)
        └─ NUM(3) (1:24)

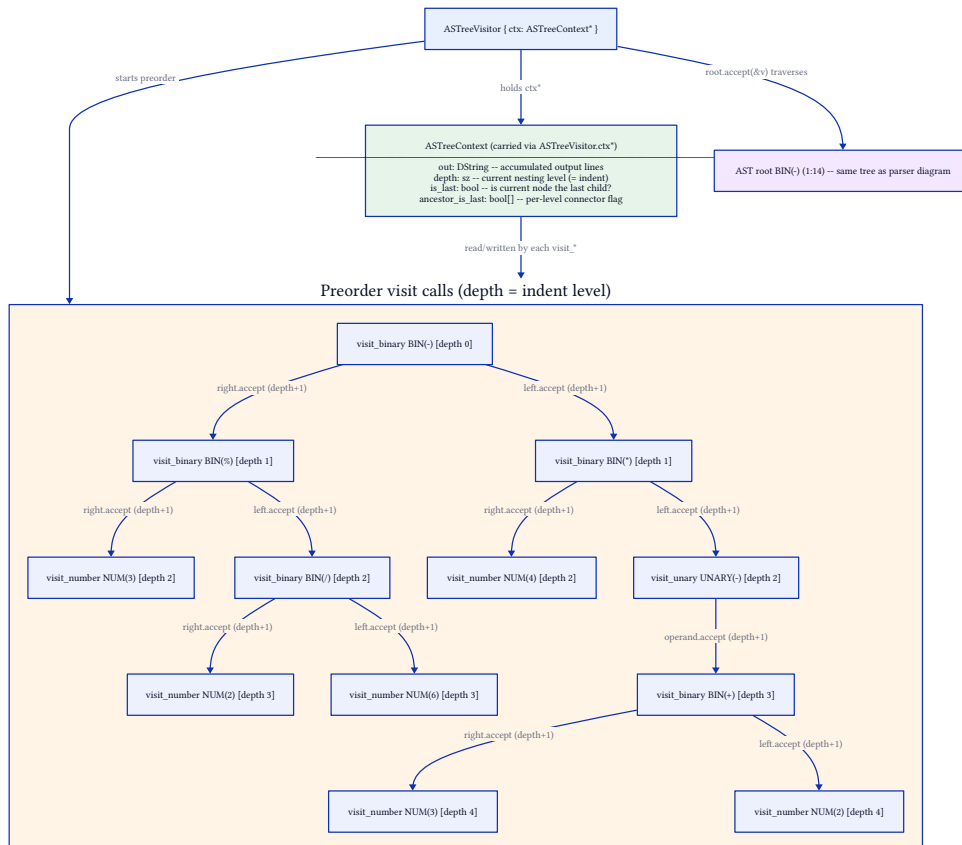
```

The coordinates in the nodes are the Ref of the corresponding tokens; they are also used in error messages.

Tree traversal context (ASTreeVisitor)

`to_tree` creates an `ASTreeVisitor` and calls `root.accept(&v)`. The visitor does not keep its state on the call stack; instead it holds it in `ASTreeContext`, referenced through `ctx*`:

- `out`: `DString` — the accumulated result (tree lines);
- `depth`: `sz` — the current nesting level, which is also the indent;
- `is_last`: `bool` — whether the current node is the last child of its parent;
- `ancestor_is_last`: `bool[]` — for each ancestor, a flag telling whether that ancestor is its parent's last child; it selects the connector | (ancestor not last) or (last).



The diagram shows the preorder traversal order. On entering a `visit_*`, the `depth` field is incremented by 1 (a child is drawn one indent deeper); on leaving it is decremented. The `is_last` flag is set before recursing into children: `false` for the left subtree and `true` for the right, and the node stores its own value in `ancestor_is_last[depth-1]` so that children can draw the correct prefix. This way the traversal context is preserved across recursive calls without being passed as a parameter.

Build and tests

```
c3c build calc3      # build (debug)
c3c build calc3 -O2 -g0 # release
c3c test calc3       # 74 unit tests (lexer, AST, parser, eval, entry point)
```